

Faculty of Computers and Information Systems

Information Retrieval

Practical Laboratory

Week 6: Inverted Index and Posting Lists

Store only the document IDs associated with each term and retrieve matches by intersecting sorted posting lists. Compare this representation with the incidence matrix.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Construct an inverted index with unique sorted posting lists.
- Calculate document frequency as posting-list length.
- Implement two-pointer posting-list intersection.
- Evaluate AND and NOT queries using postings.
- Compare postings and incidence matrices on the same collection.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

An inverted index maps each term to the documents that contain it. The dictionary provides term lookup. A posting list is the ordered list of document IDs for one term. Document frequency $DF(t)$ is the number of distinct documents containing t , equal to the length of its posting list.

Following `section-2.py`, construction first stores document IDs in a set for each term. Repeated occurrences in one document therefore add no duplicate ID. After construction, each set becomes a sorted list. The sorting order used by the index and by intersection must agree.

Two-pointer intersection compares the current IDs in two sorted lists. If they match, output the ID and advance both pointers. Otherwise advance the pointer at the smaller ID. An exhausted list ends the scan. For list lengths m and n , the scan uses at most a linear number of comparisons, $O(m + n)$.

NOT selects the loaded IDs absent from a posting list; AND uses intersection. This implementation keeps the same limited query grammar as Week 5. The incidence matrix stores T times N bits as Python list values, while postings store only occurrences of terms in documents, plus dictionary/list overhead. Both express the same Boolean membership relation.

This week adds five original teaching documents to the five supplied examples. Weeks 6-10 reuse all ten files. IDs remain `doc1` through `doc10` for continuity. String sorting places `doc10` between `doc1` and `doc2`; this is intentional and is used consistently throughout the code.

Course material: `build_index`, `intersect`, `negate`, and `run_query` in `section-2.py`. The Week 3 preprocessing policy is unchanged.

3. Python Concepts Used

- **Dictionary of sets:** Accumulate unique document IDs for each term.
- **sorted:** Convert posting sets to ordered lists.
- **while and indices:** Advance two pointers without rescanning earlier elements.
- **Dictionary get:** Return an empty list for an unknown term.
- **List membership:** Compute complement relative to all loaded document IDs.

4. Worked Example

Run this complete two-pointer trace on two short lists. It isolates the retrieval operation from tokenization and index construction.

```
left = ["doc1", "doc3", "doc4"]
right = ["doc2", "doc3", "doc4"]
i = j = 0
result = []
while i < len(left) and j < len(right):
    if left[i] == right[j]:
        result.append(left[i])
        i += 1
        j += 1
    elif left[i] < right[j]:
        i += 1
    else:
        j += 1
print(result)
```

The result is ['doc3', 'doc4']. Each pointer only moves forward. For the shipped collection, python has postings ['doc1', 'doc4'] and java has ['doc2', 'doc5']. The additional data-science document extends the machine and learning postings to include doc9.

5. Step-by-Step Practical Task

Step 1: Inspect all ten documents, noting which five have been added since Week 5.

Step 2: Use the unchanged tokenizer to accumulate a set of document IDs per term.

Step 3: Sort every posting list and inspect python, java, machine, and data.

Step 4: Complete intersect in the starter and test empty, disjoint, and overlapping lists.

Step 5: Run the three Boolean demonstrations and inspect intermediate posting lists.

Step 6: Build a Week 4-style matrix over the same ten documents and compare query results using both representations.

Execution: open a terminal in this week folder and run `python lab06_solution.py`. On Windows, `py -3` can replace `python`. All required files are included; no internet or third-party Python packages are needed.

For repeated queries, run `python lab06_solution.py --interactive`. Type quit to exit.

6. Complete Python Program

The listing below matches `lab06_solution.py` exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab06_solution.py

```
import os
import re

STOP_WORDS = {
    "a", "an", "the", "is", "in", "for", "and", "are", "with",
    "like", "of", "to", "it", "be", "used", "uses", "that", "this",
    "was", "were", "by", "from", "on", "at", "or",
}

SAMPLE_DOCUMENTS = {
    "doc1": "Python is great for data science and machine learning",
    "doc2": "Java is used for enterprise applications and web development",
    "doc3": "Machine learning uses neural networks and decision trees",
    "doc4": "Python supports data analysis with libraries like Pandas",
    "doc5": "Java and C++ are used for game development and graphics",
}
```

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_documents(folder_path):
    """Load UTF-8 .txt files; use five samples if missing or empty."""
    documents = {}
    if os.path.isdir(folder_path):
        for filename in sorted(os.listdir(folder_path)):
            filepath = os.path.join(folder_path, filename)
            if filename.endswith(".txt") and os.path.isfile(filepath):
                with open(filepath, "r", encoding="utf-8") as file:
                    documents[filename[:-4]] = file.read()
    if not documents:
        return SAMPLE_DOCUMENTS.copy()
    return documents

def tokenize(text):
    """Use the same ASCII-letter tokens as the supplied course files."""
    words = re.findall(r"[a-z]+", text.lower())
    return [word for word in words if word not in STOP_WORDS]

def parse_query(query):
    """Grammar: [NOT] term (AND [NOT] term)*; no OR or parentheses."""
    clauses = re.split(r"\bAND\b", query.strip(), flags=re.IGNORECASE)
    parsed = []
    for clause in clauses:
        match = re.fullmatch(
            r"(NOT\s+)?([a-z]+)", clause.strip(), re.IGNORECASE
        )
        if match is None:
            raise ValueError("Use: term AND term AND NOT term")
        term = match.group(2).lower()
        if term in {"and", "not", "or"}:
            raise ValueError("An operator cannot be used as a term")
        parsed.append((match.group(1) is not None, term))
    return parsed

def build_index(documents):
    index = {}
    for name, text in documents.items():
        for word in tokenize(text):
            if word not in index:
                index[word] = set()
            index[word].add(name) # A document occurs at most once.
    for word in index:
        index[word] = sorted(index[word])
    return index

def intersect(list1, list2):
    result = []
    i = 0
    j = 0
    while i < len(list1) and j < len(list2):
        if list1[i] == list2[j]:
            result.append(list1[i])
            i += 1
            j += 1
        elif list1[i] < list2[j]:
            i += 1
        else:
            j += 1
    return result

def negate(result_docs, all_docs):
    return [name for name in all_docs if name not in result_docs]

def run_query(query, index, all_docs, trace=True):
    result = None
    for negative, term in parse_query(query):

```

```

        current = index.get(term, [])
        if trace:
            print(term, "->", current)
        if negative:
            current = negate(current, all_docs)
            if trace:
                print("NOT", term, "->", current)
        if result is None:
            result = current
        else:
            result = intersect(result, current)
    return result

def main():
    import sys
    documents = load_documents(os.path.join(BASE_DIR, "docs"))
    index = build_index(documents)
    all_docs = sorted(documents)
    print("Loaded", len(documents), "documents")
    for term in ["python", "machine", "java"]:
        print(term, "->", index.get(term, []))
    if "--interactive" in sys.argv:
        print("Use term AND term AND NOT term; quit exits.")
        while True:
            try:
                query = input("Query > ").strip()
            except (EOFError, KeyboardInterrupt):
                break
            if query.lower() == "quit":
                break
            try:
                print("Matched docs:", run_query(query, index, all_docs))
            except ValueError as error:
                print(error)
    else:
        for query in ['python AND data', 'machine AND learning', 'python AND NOT java']:
            print("\nQuery:", query)
            print("Matched docs:", run_query(query, index, all_docs))

if __name__ == "__main__":
    main()

```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

```

Loaded 10 documents
python -> ['doc1', 'doc4']
machine -> ['doc1', 'doc3', 'doc9']
java -> ['doc2', 'doc5']

Query: python AND data
python -> ['doc1', 'doc4']
data -> ['doc1', 'doc4', 'doc6', 'doc7', 'doc9']
Matched docs: ['doc1', 'doc4']

Query: machine AND learning
machine -> ['doc1', 'doc3', 'doc9']
learning -> ['doc1', 'doc3', 'doc9']
Matched docs: ['doc1', 'doc3', 'doc9']

Query: python AND NOT java
python -> ['doc1', 'doc4']
java -> ['doc2', 'doc5']
NOT java -> ['doc1', 'doc10', 'doc3', 'doc4', 'doc6', 'doc7', 'doc8', 'doc9']
Matched docs: ['doc1', 'doc4']

```

These results use all ten shipped documents. Editing the collection changes the index and may change weights, ranks, and evaluation results.

8. Laboratory Exercises

1. Basic: print DF for python, machine, and data as the lengths of their postings.
2. Implementation: complete two-pointer intersection without using set intersection as the algorithm.
3. Debugging: repeat a term many times in a document. Verify that removing the construction set introduces duplicate postings.
4. Interpretation: explain why the result for machine AND learning changes after adding doc9.
5. Implementation: test NOT unknown and confirm that its universe is exactly the loaded collection.
6. Comparison: run both matrix and posting implementations on identical documents and five queries. Compare result sets before discussing storage.
7. Extension: process the shortest posting list first in a multi-term positive AND query. Count comparisons before and after.

9. Student Tasks

- Completed intersection code and tests for empty, disjoint, identical, and overlapping lists.
- Selected index entries, their document frequencies, and traces for the default queries.
- A short matrix/postings comparison using the same ten-document collection.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. What does an inverted index map from and to?
2. Why are sets used during construction?
3. Why must posting lists be sorted for two-pointer intersection?
4. How does document frequency differ from term frequency?
5. Why must two representations use the same collection when comparing results?

11. Challenge Exercise

Optional: Implement sorted posting-list union and difference with two pointers. Test them independently before adding more query operators.